FICHA PRÁTICA LABORATORIAL 03 – AI-POWERED SOFTWARE ENGINEERING

Objetivos

- Implementar o MVP com apoio de AI Assistants ou Coding Agents.
- Aplicar as decisões de arquitetura, dados, API e interface já documentadas.
- Desenvolver por incrementos e verificar cada funcionalidade.
- Investigar problemas, rever alterações e manter o projeto versionado em Git.

ÂMBITO FUNCIONAL

Área	Comportamento mínimo
Autenticação	Iniciar e terminar sessão e consultar o utilizador autenticado.
Projetos	Criar, listar, consultar, editar e eliminar projetos autorizados.
Tarefas	Criar, consultar, editar e eliminar tarefas de um projeto, com estado e prioridade.
Responsáveis	Associar vários utilizadores elegíveis a uma tarefa e remover atribuições.
Comentários	Adicionar e listar comentários de uma tarefa, com autor e data.
Pesquisa e filtros	Pesquisar projetos pelo nome e filtrar tarefas por estado e prioridade.
Painel inicial	Apresentar projetos acessíveis e contagens de tarefas por estado, com navegação para o detalhe.

Este âmbito deve ser conciliado com o MVP aprovado nos documentos anteriores. Registar e resolver diferenças no Exercício 1 antes de implementar.

Para simplificar o acesso, podem ser usadas contas de demonstração criadas por seed. O registo público e a recuperação de palavra-passe são extensões opcionais.

PREPARAÇÃO E PLANEAMENTO

Exercício 1 Consolidar o plano de implementação

Objetivo: transformar a documentação existente num plano executável e coerente.

1. Abrir o repositório Matriz e rever AGENTS.md, README.md, docs/requirements.md, docs/user-stories.md, docs/constraints.md, docs/architecture.md, docs/database.md, o OpenAPI e o wireframe.
2. Confirmar a linguagem, as frameworks e a base de dados escolhidas na Aula 02. Se a decisão estiver pendente, selecionar uma solução simples e registá-la em docs/architecture.md.
3. Consolidar as regras de acesso: quem gere projetos, quem pode consultar ou alterar tarefas, quem pode comentar e quem pode atribuir responsáveis. Respeitar o modelo aprovado e resolver ambiguidades com o formador.
4. Completar o contrato da API para as operações necessárias. A Ficha 02 exigia apenas duas operações; o exemplo completo também necessita de completar o fluxo de autenticação e a seleção dos utilizadores elegíveis, se for utilizado.
5. Criar docs/implementation-plan.md com a sequência de incrementos, ficheiros ou componentes previstos, critérios de aceitação e formas de verificação. Atualizar o estado do plano ao longo da implementação.

Resumo exemplar de prompt de apoio ao planeamento

Analisa as instruções e os artefactos da Matriz neste repositório.
Compara o MVP com o âmbito desta ficha e identifica lacunas, contradições e decisões técnicas ainda pendentes.
Propõe um plano incremental que respeite a stack escolhida.
Para cada incremento, indica componentes, dependências, critérios de aceitação e verificações.
Distingue decisões confirmadas de propostas. Não implementes ainda.

Exercício 2 Preparar a aplicação e a persistência

- Criar a estrutura do projeto e instalar as dependências necessárias, seguindo a arquitetura escolhida.
- Configurar a ligação à base de dados, as variáveis de ambiente e um ficheiro de exemplo com valores fictícios. Manter credenciais reais fora do repositório.
- Criar o esquema e as migrations, ou um procedimento equivalente reproduzível, para utilizadores, projetos, tarefas, comentários e atribuições.
- Preparar dados de demonstração reproduzíveis, com pelo menos dois utilizadores e projetos que permitam testar acessos distintos. A inicialização não deve apagar dados em cada arranque.
- Confirmar que a aplicação arranca e que o acesso à base de dados funciona. Registar os comandos no README.md.

Resultado esperado: projeto executável, esquema criado e plano de implementação revisto.

AUTENTICAÇÃO E PROJETOS

Exercício 3 Implementar o acesso à aplicação

Objetivo: identificar o utilizador e aplicar as regras de acesso no Backend.

1. Implementar o início de sessão, a consulta do perfil e o fim de sessão, de acordo com o mecanismo de autenticação escolhido. Documentar o ciclo da sessão ou do token no contrato e no README.
2. Se existirem palavras-passe locais, guardar apenas hashes através de uma biblioteca adequada. Usar contas e dados fictícios na demonstração.
3. Obter a identidade do utilizador a partir da autenticação. Inferir no Backend o proprietário do projeto e o autor dos comentários, em vez de aceitar esses valores como prova de identidade enviada pelo cliente.
4. Proteger as operações e verificar a autorização sobre cada projeto e respetivas tarefas, comentários e atribuições. Aplicar as regras também a pedidos diretos à API.
5. Verificar credenciais válidas e inválidas, pedidos sem autenticação e acessos com dois utilizadores diferentes.

A interface deve encaminhar o utilizador para o início de sessão quando necessário e apresentar uma mensagem compreensível quando a sessão deixa de ser válida.

Exercício 4 Implementar a gestão de projetos

Objetivo: disponibilizar o primeiro percurso completo entre interface, API e base de dados.

- Implementar a criação, listagem, consulta, edição e eliminação de projetos, respeitando os pedidos, respostas e códigos HTTP do contrato.
- Validar o nome e a descrição segundo o OpenAPI adotado. No exemplo da Aula 02, o nome tem entre 3 e 100 caracteres e a descrição admite até 1000.
- Associar o projeto ao utilizador autenticado e limitar a listagem e as operações aos projetos que lhe são acessíveis.
- Permitir a pesquisa pelo nome e cumprir a paginação, caso esteja definida no contrato.
- Pedir confirmação na interface antes de eliminar e aplicar a regra documentada para os registos dependentes, evitando dados órfãos.

Primeiro incremento a concluir: iniciar sessão, criar um projeto, vê-lo na listagem e voltar a consultá-lo após recarregar a página e reiniciar o Backend.

Resultado esperado: autenticação funcional e gestão de projetos integrada com persistência real.

Tarefas e colaboração

Exercício 5 Implementar a gestão de tarefas

Objetivo: gerir o trabalho de cada projeto através de operações reais na API.

- Implementar a criação, listagem, consulta, edição e eliminação de tarefas associadas a um projeto existente e autorizado.
- Permitir alterar o estado e a prioridade e filtrar a listagem por esses campos. Respeitar a paginação prevista no contrato.
- Validar os dados no Backend e apresentar os erros relevantes na interface. Rejeitar nomes ou títulos compostos apenas por espaços.
- Atualizar listagens e contagens após cada operação, sem exigir que o utilizador recarregue a página.

Referência de campos do exemplo OpenAPI da Aula 02

Campo	Regra
Título	Obrigatório, entre 3 e 150 caracteres.
Descrição	Até 2000 caracteres.
Estado	TODO, IN_PROGRESS ou DONE. Valor inicial TODO.
Prioridade	LOW, MEDIUM ou HIGH. Valor inicial MEDIUM.

A interface pode traduzir os estados e prioridades para português, preservando os valores técnicos acordados com a API.

Exercício 6 Implementar responsáveis e comentários

- Disponibilizar a seleção de utilizadores elegíveis segundo as regras de acesso. Associar vários responsáveis à mesma tarefa e permitir remover cada atribuição.
- Impedir a repetição da associação entre a mesma tarefa e o mesmo utilizador. Validar a existência e a elegibilidade do utilizador no Backend.
- Adicionar e listar comentários de uma tarefa, apresentando o autor e a data. No exemplo OpenAPI, o conteúdo tem entre 1 e 2000 caracteres e não deve aceitar apenas espaços.
- Obter o autor e a data no Backend. Confirmar que os comentários e as atribuições permanecem disponíveis após reabrir a tarefa.

Para cada funcionalidade, pedir primeiro a análise e o plano. Após a validação, implementar um incremento, executar as verificações relevantes e rever o diff antes de continuar.

Resultado esperado: tarefas persistentes com estado, prioridade, responsáveis e comentários funcionais.

INTERFACE E INTEGRAÇÃO

Exercício 7 Implementar o Frontend da Matriz

Objetivo: permitir utilizar todas as funcionalidades do MVP através de uma interface coerente com o wireframe.

Organizar a navegação pelos seguintes ecrãs ou componentes equivalentes:

Ecrã	Conteúdo e ações
Início de sessão	Formulário, validação e mensagens de erro.
Painel inicial	Utilizador autenticado, projetos acessíveis e contagens de tarefas por estado.
Projetos	Listagem e pesquisa, criação e acesso ao detalhe, edição e eliminação.
Detalhe do projeto	Dados do projeto, listagem de tarefas, filtros e formulário de criação.
Detalhe da tarefa	Dados, edição, estado, prioridade, responsáveis, comentários e eliminação.

- Reutilizar componentes e estilos, mantendo os formulários e as mensagens consistentes. Adaptar o layout a diferentes larguras de ecrã.
- Ligar as ações à API real e centralizar a configuração do endereço do Backend. O endereço ilustrativo do exemplo OpenAPI não corresponde a um serviço ativo.
- Tratar os estados de carregamento, lista vazia, sucesso e erro. Preservar os dados introduzidos quando uma submissão falha.
- Evitar submissões repetidas enquanto um pedido está em curso. Usar a resposta confirmada pela API para atualizar a interface, sem duplicar registos.
- Sincronizar listagens, filtros e contagens após criar, editar ou eliminar. Os totais devem refletir o conjunto de dados relevante, e não apenas a página visível.
- Verificar o percurso completo com a API e a base de dados. Retirar os mocks das funcionalidades concluídas.

Prompt de apoio à implementação

Implementa o próximo incremento do plano validado da Matriz.
Consulta AGENTS.md, a User Story, a arquitetura, o modelo de dados, o contrato da API e o wireframe relevantes.
Respeita a stack escolhida e reutiliza os componentes existentes.
Inclui validação, autorização e os estados da interface aplicáveis.
Mantém as alterações limitadas ao incremento e executa as verificações previstas. Apresenta o diff, os resultados reais e os problemas que ficarem por resolver.

Resultado esperado: MVP utilizável na interface, com todas as operações ligadas ao Backend.

VERIFICAÇÃO E RESOLUÇÃO DE PROBLEMAS

Exercício 8 Verificar o MVP

Objetivo: demonstrar que a aplicação cumpre os requisitos e conserva os dados. Registar em docs/verification.md o resultado esperado, o resultado observado e a evidência de cada cenário.

Caso	Cenário	Resultado a confirmar
01	Acesso à aplicação	Credenciais válidas permitem entrar; inválidas e pedidos sem autenticação são tratados.
02	Gestão de projetos	Criar, pesquisar, editar e eliminar funciona de acordo com as permissões.
03	Persistência de tarefas	A tarefa aparece sem recarregar e mantém-se após reiniciar o Backend.
04	Validação de dados	Campos obrigatórios vazios ou só com espaços e valores inválidos são rejeitados.
05	Estado e prioridade	A edição atualiza a tarefa, os filtros e as contagens de forma coerente.
06	Responsáveis	Associar e remover funciona; a mesma associação não fica duplicada.
07	Comentários	O conteúdo, o autor autenticado e a data ficam guardados e visíveis.
08	Autorização na API	Outro utilizador não consegue aceder ou alterar recursos sem permissão, mesmo mudando os identificadores no pedido.
09	Eliminação e erros	A eliminação respeita as dependências; erros da API não geram mensagens de sucesso nem perda dos dados do formulário.

- Criar e executar pelo menos três testes automatizados relevantes: uma regra de validação, um cenário de autorização e uma operação de criação com consulta posterior à persistência.
- Registar os comandos executados, resultados e limitações. Distinguir verificações manuais de testes automatizados e identificar o que não foi possível executar.
- Quando surgir um bug, reproduzir, recolher evidência, investigar a causa e aplicar uma correção limitada. Acrescentar um teste de regressão adequado.

Prompt de apoio à investigação

Investiga o problema descrito, usando os passos de reprodução, o resultado esperado, o observado e a evidência fornecida.

Localiza o código envolvido e distingue hipóteses de conclusões.

Explica como confirmar a causa e propõe uma correção limitada.

Não alteres ficheiros antes de apresentar o diagnóstico.

Resultado esperado: verificações registadas, testes executados e problemas conhecidos identificados.

REVISÃO E ENTREGA

Exercício 9 Rever e preparar a entrega

1. Rever o diff com apoio da IA e confirmar o cumprimento dos critérios de aceitação. Inspecionar especialmente permissões, validações, tratamento de erros e alterações fora do âmbito.
2. Identificar uma oportunidade de melhoria do código. Se a refatoração trouxer benefício, aplicá-la num passo separado e verificar que o comportamento foi preservado. Registar a decisão.
3. Atualizar o OpenAPI e os documentos afetados para refletirem a implementação real. Atualizar AGENTS.md com os comandos de execução e verificação relevantes.
4. Completar o README.md com requisitos, instalação, configuração, criação do esquema, seed, contas fictícias de demonstração, arranque e execução dos testes.
5. Registar em docs/verification.md três exemplos breves de utilização da IA: objetivo do pedido, decisão tomada pelo programador e forma de verificação. Incluir uma sugestão corrigida ou rejeitada, se tiver ocorrido.
6. Versionar o trabalho em commits com mensagens claras. Confirmar que os ficheiros necessários estão incluídos e que os passos do README permitem executar o projeto a partir de uma cópia limpa.

ENTREGÁVEIS

- Código completo do Frontend e do Backend, com testes e ficheiros de dependências.
- Esquema ou migrations, dados de demonstração e exemplo de configuração.
- README.md, AGENTS.md e contrato OpenAPI atualizados.
- Documentação anterior coerente com a solução, docs/implementation-plan.md e docs/verification.md.

A estrutura do código pode variar com a stack adotada. Os artefactos devem estar organizados no mesmo repositório Matriz iniciado na Ficha 01.

CRITÉRIOS DE CONCLUSÃO

☐ A aplicação arranca seguindo as instruções do README.

☐ O MVP permite gerir projetos, tarefas, responsáveis e comentários através da interface.

☐ A autenticação e a autorização funcionam também em pedidos diretos à API.

☐ Os dados persistem após recarregar a interface e reiniciar o Backend.

☐ A interface trata carregamento, ausência de dados, sucesso e erro.

☐ O contrato corresponde à implementação e as verificações têm resultados registados.

☐ O código foi revisto e a versão entregue está identificada em Git.

Bom trabalho! ☺